

PROMETHEUS

ARCHITECTURAL MAP v8.1

The Locked Execution Specification

Local-First Doctrine + Codex Amendments

Version: 8.1 | **Status:** LOCKED | **Date:** 2026-06-20

Sprint: 14 Days | **Target:** 90-Second Vertical Shorts (1080×1920)

Stack: R3F + Remotion + FFmpeg | **Render:** Sequential Local CLI

Governor: Judgment Layer | **Variation:** Candidate Genomes + Replay Ledger

Approved by: Gemini + Kimi + Codex + Human

PROMETHEUS ARCHITECTURAL MAP v8.1

The Locked Execution Specification — Local-First Doctrine + Codex Amendments

Version: 8.1

Status: LOCKED — Approved for Build

Date: 2026-06-20

Sprint Duration: 14 Days

Target: 90-Second Vertical Shorts (1080×1920)

Stack: R3F + Remotion + FFmpeg

Render Mode: Sequential Local CLI (No distributed infrastructure for MVP)

Governor: Judgment Layer (not schema)

Variation Engine: Candidate Treatment Genomes + Replay Ledger + Explicit Variation Key

TABLE OF CONTENTS

1. Executive Summary
2. The Paradox Resolution
3. The Doctrine (Non-Negotiable Axioms)
4. The Lock v8.1 (Frozen Decisions)
5. The Governor Correction (NEW)
6. The Variation Contract (NEW)
7. The Local-First Architecture
8. The Replay Ledger (NEW)
9. The Candidate Set Rule (NEW)
10. The Complete Failure Node Map
11. The 90-Second MVP Specification
12. The Asset Contract (NEW)
13. The Render Entry Contract (NEW)
14. The 2-Week Sprint Plan
15. The Build Order
16. Performance & Scaling Math

- 17. The Risk Register
- 18. The Agent Protocol
- 19. The Go/No-Go Gates
- 20. The Phase 2 Hook
- 21. The Determinism Contract
- 22. The FFmpeg Specification
- 23. The Physics Engine Specification
- 24. The Font Pipeline Specification
- 25. The Audio Mixing Specification
- 26. The Director Rules & Heuristics
- 27. The Sequence Memory State Machine (NEW)
- 28. The Prompt Governance Rules (NEW)
- 29. The Evidence Preservation Path (NEW)
- 30. The CI/CD Specification
- 31. The Review Floor (NEW)
- 32. The Verified Assumptions
- 33. Appendix A: Glossary
- 34. Appendix B: Plain-English Breakdown

1. EXECUTIVE SUMMARY

Prometheus is a Cinematic Attention Orchestration System. It is not a traditional video editor. It is an engine that listens to audio, reads transcripts, feels speaker energy, and mathematically dictates where cuts happen, how text bounces, and when sound effects fire.

The v8.1 architecture is a local, heavy-duty engine that runs entirely on your machine. You feed it raw footage, and it spits out a highly kinetic, 90-second vertical Short.

What v8.1 Adds Over v8

Addition	Rationale
Governor Correction	Schema is the contract. Judgment Layer is the governor.
Variation Contract	Defines what stays invariant vs. what varies across re-uploads.
Replay Ledger	Local persisted memory for anti-repetition and creator continuity.
Candidate Set Rule	Multiple Treatment Genomes per input, selected by Judgment Layer.

Addition	Rationale
Asset Contract	Explicit browser-facing vs. FFmpeg-facing media references.
Render Entry Contract	Joseph-specific render entry to avoid bundling unrelated app code.
Prompt Governance	Rules preventing prompts from breaking infrastructure or determinism.
Sequence Memory State Machine	Explicit states (calm, building, saturated, recovering) instead of prose.
Evidence Preservation	Persist candidates, rejected candidates, and review verdicts for future learning.
Review Floor	Measurable acceptance gates replacing the "85% Joseph" claim.

2. THE PARADOX RESOLUTION

The Problem

We designed a cloud-native distributed pipeline (BullMQ, Redis, parallel chunking, GPU worker farms, monitor processes) and then declared the MVP would run locally on user laptops. These are mutually exclusive. A MacBook Air cannot run 9 concurrent headless Chrome instances. A Windows machine does not have Redis installed.

The Resolution

Cut the distributed infrastructure entirely for MVP. The system is a local CLI tool that produces a 90-second vertical Short. It runs sequentially. It uses the local GPU if available, or software rendering if not. It does not need a queue, a worker farm, or a monitor process.

Why This Is Correct

Reason	Explanation
Render Time Acceptable	90 sec on GPU = ~45 sec. On software = ~5 min. Both acceptable for MVP testing.
Creative Risk Dominates	Director quality, physics feel, and audio sync are 100x more important than infrastructure.
Foundation First	We cannot test distributed rendering until the single-machine render is perfect.
Clean Phase 2 Hook	<code>`renderFromManifest()`</code> signature is preserved for BullMQ wrapping.
Agent Feasibility	Distributed debugging causes agents to hallucinate. Local sequential is debuggable.

What Was Cut

- BullMQ / Redis
- Parallel chunking
- FFmpeg stitch

- Monitor process
- Cloud backend
- Load balancer
- Horizontal scaling

What Was Preserved

- R3F + Remotion stack
- Determinism contract
- Bundle cache
- CPU-only physics engine
- Rule-based Director
- Audio ducking
- SFX system

3. THE DOCTRINE (NON-NEGOTIABLE AXIOMS)

These are axioms. Violating any of them invalidates the architecture.

#	Axiom	Rationale	Violation Consequence
1	**R3F + Remotion is the stack.**	`AGENTS.md` Rule 1. The pixel engine is the core of the experience.	Not allowed. All the core code is in Remotion.
2	**Determinism is mandatory.**	Same `seed` + same `manifest` = pixel engine output. Enables A/B testing and trust.	Identical inputs must produce identical outputs.
3	**90 seconds is the hard MVP cap.**	Eliminates scene detection, narrative sequencing, hyper-ambitions, etc. No pre-rendering.	Scene must be ready in 90 seconds for Phase 4 intro.
4	**Audio-first, not transcript-first.**	The cut happens because of the audio cut, not the transcript cut. It's like a slate show with a cut.	Audio must drive the timeline, not the transcript.
5	**Rule-based Director for Phase 1.**	No IRL, no multi-agent, no neural policy. We ship it for 6 months while chasing theoretical RL.	Director must be rule-based for Phase 1.
6	**Local CLI for MVP.**	Runs on developer machine. No cloud dependency. No AWS. If we build locally, we can test without AWS.	Must be able to run locally for MVP.
7	**The Judgment Layer is the governor.	The schema enforces shape. The Judgment Layer governs quality, behavior, and cost.	Schema must govern quality, behavior, and cost.
8	**Variation is explicit, not hidden.**	Same source re-upload must vary via explicit keys, not hidden randomness.	Variation must be explicit and controllable.

4. THE LOCK v8.1 (FROZEN DECISIONS)

These are decided. No agent may reopen them without human approval.

Decision	Value	Rationale
Stack	R3F + Remotion	Non-negotiable per `AGENTS.md`
Duration cap	90 seconds	Market is Shorts. Hard limit at schema level.
Resolution	**1080×1920 vertical**	TikTok/Instagram/Reels native.
FPS	30 CFR	No VFR. No negotiation.
Determinism	Mandatory	Same seed = same pixels. CI enforces.
Director	Rule-based, 3 profiles	No IRL. No multi-agent.
Physics	CPU-only, Node.js	WebGL receives final matrices only.
Font engine	**DOM Canvas fallback PRIMARY**	`opentype.js` is stretch goal only.
Audio	Band-pass sidechain (300Hz–3000Hz)	Demucs deferred to Phase 2.
Render	**Sequential, single-chunk**	No parallel chunking for MVP.
Queue	**None**	No BullMQ, no Redis, no monitor.
Bundle	Cached (SHA256 of src)	Saves 42s per render.
SFX	8 categories × 5 variations = 40 cues	Seeded selection.
Text placement	Upper-middle heuristic (top 40%)	Face overlap acceptable in <5% of frames.
Breathe	Visual-only (slow push-in + music swell)	No dead air.
Candidates per input	2–6 Treatment Genomes	Selected by Judgment Layer.
Variation key	`upload_instance_id` + `retry_index`	Explicit, not hidden.
Governor	Judgment Layer	Schema is contract, not authority.
Memory	Local Replay Ledger (SQLite/JSONL)	Not cloud. Not in-memory only.

5. THE GOVERNOR CORRECTION (NEW)

What Was Wrong

The v8 document stated: "The schema is the governor."

This conflicts with the repo glossary. A schema can enforce shape and invariants. It cannot govern editorial quality, novelty, restraint, or anti-repetition. If you make the schema the governor, you overburden validation and underbuild the actual decision module.

The Correction

Role	Responsibility	Analogy
Schema	Interface contract. Enforces shape, types, required fields.	The constitution
Judgment Layer	Governor. Enforces quality floor, novelty, taste, and repetition.	The supreme court
Top-Level Planner	Generates candidate Treatment Genomes.	The legislature
Director Adapter	Concrete implementation. Currently houses both planning and judgment.	The executive branch in Phase 1.

Phase 1 Implementation

For Phase 1, the concrete joseph-director.ts module may implement both planning and judgment. However, the logical seams must be named:

- planCandidates() — generates N candidate Treatment Genomes
- judgeCandidates() — applies quality floor, anti-repetition, similarity veto
- selectCandidate() — final selection using variation key

This preserves locality now without losing the future seam.

6. THE VARIATION CONTRACT (NEW)

The Problem

The v8 document talked strongly about determinism and quality, but did not define the variation contract precisely. If the same source video is re-uploaded, what stays the same? What changes? Who decides?

The Contract

Invariant Set (Always Same)

- Observation Snapshot (transcript, beats, onsets, energy curve)
- Source-derived visual facts (duration, resolution)
- Hard production constraints (90-sec cap, vertical resolution, CFR)
- Prompt fingerprint (if prompt strongly biases doctrine)

Variable Set (Allowed to Change)

- Chosen Treatment Genome (from candidate set)
- Text timing selections (within ± 50 ms tolerance)
- Camera directive placement (within ± 100 ms tolerance)

- Transition choice (hard vs. zoom_blur)
- SFX assignment (which cue, which variation)
- Font selection (from the 5-hero pool)

Variation Key

```
interface VariationKey {
  sourceFingerprint: string;    // SHA256 of source video
  promptFingerprint: string;    // Hash of prompt text
  uploadInstanceId: string;     // UUID per upload
  retryIndex: number;           // 0 for first render, increments on retry
}
```

Determinism Definition

- **Same key** (sourceFingerprint + promptFingerprint + uploadInstanceId + retryIndex) → **identical output**
- **Different uploadInstanceId** or **different retryIndex** → **bounded variation family**, not clone

Enforcement

The Judgment Layer compares the selected candidate against the Replay Ledger. If similarity to a prior output for the same source exceeds the threshold, the candidate is vetoed and the next-best candidate is selected.

7. THE LOCAL-FIRST ARCHITECTURE

```
[User runs CLI]
■■■ npx tsx scripts/test-joseph.ts --full
■■■ [Director/Planner] generates N candidate Treatment Genomes
■■■ [Judgment Layer] selects 1 candidate via quality floor + Replay Ledger
■■■ [Audio] band-pass ducking + SFX placement
■■■ [Physics Engine] pre-computes transforms (Float32Array)
■■■ [Worker] renders sequentially via Remotion + R3F
■   ■■■ Bundle cache (compute once, reuse)
■   ■■■ GPU probe (angle → swangle fallback)
■   ■■■ Chrome cleanup (5-min timeout, SIGKILL)
■■■ [Replay Ledger] persists output metadata for future anti-repetition
■■■ [Output] single MP4, 1080×1920, 30fps
```

Component Interaction

Component	Input	Output	Responsibility
Top-Level Planner	Observation Snapshot, prompt, profile	N candidate Treatment Genomes	Generates editorial possibilities
Judgment Layer	Candidates, Replay Ledger, variation key	Selected Treatment Genome	Applies quality floor, novelty, taste
Director Adapter	Source video, transcript, audio, seed, final manifest	Final manifest	Concrete implementation of Planner + Judgment
Physics Engine	Manifest text overlays, duration, seed	`Float32Array` + `Uint8Array`	Pre-computed per-frame transforms
Audio Mixer	Mixed audio, manifest SFX cues	Mixed MP3 with ducking	FFmpeg `filter_complex`


```
id TEXT PRIMARY KEY,          -- UUID
source_fingerprint TEXT NOT NULL, -- SHA256 of source video
prompt_fingerprint TEXT,      -- Hash of prompt text
upload_instance_id TEXT NOT NULL,
retry_index INTEGER DEFAULT 0,
profile TEXT NOT NULL,        -- joseph_aggressive | cinematic | minimal
chosen_genome TEXT NOT NULL,   -- JSON of selected Treatment Genome
rejected_genomes TEXT,        -- JSON array of rejected candidates
planner_audit TEXT,           -- JSON of compact Planner Audit
similarity_hash TEXT,          -- Perceptual hash / summary vector
quality_score REAL,           -- Judgment Layer score
failure_tags TEXT,            -- Comma-separated Failure Taxonomy tags
created_at TEXT               -- ISO timestamp
);

CREATE INDEX idx_source ON replay_ledger(source_fingerprint);
CREATE INDEX idx_prompt ON replay_ledger(prompt_fingerprint);
CREATE INDEX idx_created ON replay_ledger(created_at);
```

Why Not Pattern Memory?

Pattern Memory in the glossary is a reusable outcome ledger for known motion/layout patterns across runs. That is broader and more semantically loaded.

Replay Ledger is the smaller seam for:

- Same-source anti-repetition
- Retry diversification
- Creator continuity thresholds

Later, Replay Ledger can inform Pattern Memory or Creator Taste Memory.

Comparison Thresholds

Comparison Type	Threshold	Action if Exceeded
Same source, same prompt	Similarity > 0.85	Veto candidate. Select next-best.
Same source, different prompt	Similarity > 0.70	Veto candidate. Select next-best.
Same creator, different source	Similarity > 0.60	Warning. Allow with style continuity note.

Location

~/.prometheus/replay-ledger.sqlite (local user directory)

9. THE CANDIDATE SET RULE (NEW)

The Problem

If the Director emits only 1 Treatment Genome per input, novelty is weak. If it emits too many, latency rises.

The Rule

Phase 1 emits a small bounded candidate set:

Profile	Candidate Count	Rationale
Aggressive	4–6 candidates	High complexity, more valid paths
Cinematic	3–4 candidates	Medium complexity
Minimal	2–3 candidates	Low complexity, fewer valid paths

Selection Process

- 1. **Generate:** Top-Level Planner emits N candidates
- 2. **Filter:** Judgment Layer removes candidates below quality floor
- 3. **Veto:** Judgment Layer removes candidates too similar to Replay Ledger entries
- 4. **Select:** Final selection using variation key (seeded from upload_instance_id + retry_index)

Why This Matters

This is the cheapest path to real re-upload variation without IRL. It gives you "LLM retry-style variation" — same source, different output, bounded by quality and taste.

10. THE COMPLETE FAILURE NODE MAP v8.1

#	Node	Severity	Problem	Solution	Phase	Owner
1	**Render Bottleneck**	Existential	`swangle` software render is slow @ 90 FPS in 1080p	Render to lower resolution, use GPU	Phase 1	Worker
2	**Asset Pipeline**	Systemic	`file:///` blocked in Chromium Assets, explicit `browserUrl` vs `file`	Use `chrome://` or `chrome-extension://`	Phase 1	Worker
3	**Typography Physics**	Differentiator	Text "appears" instead of "disappears" in physics engine	Use physics engine for text placement	Phase 2	Physics + Director
4	**Audio-First Cuts**	Core	Cuts are near beats, not on beats	Use beat detection to place cuts on beats	Phase 2	Director places cuts ON beats.
5	**Face-Aware Placement**	Polish	Text covers speaker's face	Upper-middle heuristic: text in top 40% of frame	Phase 2	Director Heuristic, not guarantee.**
6	**The "Breathe Beat"**	Aesthetic	Hyper-kinetic exhaustion, no breathing space	Use breathing space, push-in + music sync	Phase 2	Director No dead air.
7	**Source Separation**	Systemic	Cannot distinguish vocal from instrumental	Use source separation (e.g. Demucs)	Phase 2	Audio Phase 2.
8	**Font Pipeline**	Isolated	577 fonts untested. TTF/OTF fails fallback	Use fallback font (e.g. Arial)	Phase 2	Font Pipeline
9	**Schema Future-Proofing**	Systemic	Schema lacks visual analysis field	Add visual analysis field to schema	Phase 2	Schema
10	**The "Irony Cut"**	Phase 3	Cannot cut on silence, requires context	Requires context, humor detection	Phase 3	Director

#	Node	Severity	Problem	Solution	Phase	Owner
11	**Chrome Cleanup**	Systemic	Headless Chrome process hangs after 100s. SIGKILL on process.	Process hang detection.	Worker	Worker
12	**Color Space Mismatch**	Cosmetic	R3F sRGB vs FFmpeg sRGB. 709 flags in FFmpeg.	FFmpeg sRGB.	Worker	Worker
13	**Transcript Confidence**	Isolated	Whisper hallucinates. Min word duration 150ms. Stop-word filter.	Min word duration 150ms. Stop-word filter.	Director	Director
14	**Beat False Positives**	Isolated	Breath sounds, chair wheels trigger false beats.	Voice trigger false beats. Cross-reference audio beats + onsets.	Audio	Audio
15	**Preview vs Render**	Systemic	Preview uses GSAP. Render physics springs. Preview uses `useCustomFrame` not `requestAnimationFrame`.	Render physics springs. Preview uses `useCustomFrame` not `requestAnimationFrame`.	Director	Director
16	**Creative Drift**	Isolated	Seeded randomness. Onset to beat divergence, contrast ratio, no director zoom simultaneously.	Onset to beat divergence, contrast ratio, no director zoom simultaneously.	Director	Director
17	**Export Codec**	Cosmetic	H.264 large files. Platform 264 is encoded for M3P. Platforms expect	Platform 264 is encoded for M3P. Platforms expect	Worker	Worker
18	**Song/Video Mismatch**	Systemic	Music shorter or longer than video. Fade at video end.	Audio track boundaries. Fade at video end.	Audio	Audio
19	**Prompt Injection**	Existential	Agent introduces `Math.random(), GSAP.timeline()`. Deterministic.	Math.random(), GSAP.timeline(). Deterministic.	DevOps	DevOps
20	**SFX Repetition**	Isolated	8 cues repeated = 8 categories × 5 variations = 40 cues. Seed	8 categories × 5 variations = 40 cues. Seed	Director	Director
21	**Intensity Budget**	Aesthetic	90 sec of max intensity. Sequential primary tracks cumulative energy. Director breathes at exhaustion.	Sequential primary tracks cumulative energy. Director breathes at exhaustion.	Director	Director
22	**OS Font Divergence**	Existential	Troika uses OS font. On macOS, cross-platform font divergence. Cannot bypass.**	On macOS, cross-platform font divergence. Cannot bypass.**	Director	Director
23	**JSON OOM**	Existential	Passing 50MB JSON. Space for 12.5MB. OOM on object literals.	Space for 12.5MB. OOM on object literals.	Physics	Physics
24	**GC Pause**	Systemic	Object allocation in physics. Physical types arrays. Zero `new` in physics.	Physical types arrays. Zero `new` in physics.	Physics	Physics
25	**Agent Scope Creep**	Existential	Agent hits bug, rewrites code. No agent touches >1 vertex.	Code isolation. No agent touches >1 vertex.	Physics	Physics
26	**FFmpeg Macro-block**	Cosmetic	`-preset fast` causes pipeline modification.	Preset fast modification.	Worker	Worker
27	**Vertical Aspect Ratio**	Systemic	1080×1920 may crop video. Preserve aspect ratio.	Video frame crop, preserve aspect ratio.	Director	Director
28	**Bundle Invalidation**	Systemic	Cache serves stale code. Deployed key: SHA256 of `remote/src/`.	Cache serves stale code. Deployed key: SHA256 of `remote/src/`.	Worker	Worker
29	**Schema/Runtime Diff**	Systemic	Source schema changes. Build dist are not synchronized.	Source schema changes. Build dist are not synchronized.	DevOps	DevOps
30	**Placement Validation**	Isolated	No way to measure head placement. Reltags to Review Surface.	Head placement. Reltags to Review Surface.	Director	Director
31	**Variation Fakeness**	Existential	Re-upload yields identical. Replay Ledger + explicit validation.	Replay Ledger + explicit validation.	Director	Director
32	**Governor Ambiguity**	Existential	Same module plans, judges logical. 1Planner → Judgment.	Judges logical. 1Planner → Judgment.	Director	Director
33	**Prompt Sabotage**	Existential	One bad prompt breaks. Prompt Governance Rules.	Prompt Governance Rules.	Architecture	Architecture
34	**Evidence Loss**	Systemic	No artifacts preserved. Evidence Preservation Path: persist candidate, verdicts.	Evidence Preservation Path: persist candidate, verdicts.	Director	Director

11. THE 90-SECOND MVP SPECIFICATION v8.1

Video Output

Parameter	Value	Rationale
Duration	≤ 90 seconds (hard cap)	Shorts market. Eliminates narrative complexity.

Parameter	Value	Rationale
Resolution	1080 × 1920	Vertical native for TikTok, Instagram Reels, YouTube Shorts.
FPS	30	
Frame Rate Mode	CFR (Constant Frame Rate)	
Codec	H.264	Universal compatibility. All platforms accept it.
CRF	18	High quality. File size acceptable for 90 sec.
Preset	`medium`	Balance of speed and quality. `fast` causes macro-blocking.
Color Space	BT.709	Standard for web video.
Pixel Format	yuv420p	Maximum compatibility.

Audio Output

Parameter	Value	Rationale
Ducking	Band-pass sidechain (300Hz–3000Hz)	Isolates vocal frequencies. Minimizes drum false positives.
Music Level	-18 dB when voice detected	
Voice Level	0 dB (reference)	
SFX Level	-10 to -12 dB	Present but not overwhelming.
Target LUFS	-14	Platform standard for Shorts.
Sample Rate	48 kHz	Video standard.
Bitrate	192 kbps	Transparent for speech + music.

Director Output

Parameter	Aggressive	Cinematic	Minimal
Hook Cuts	5+	3–4	2–3
Body Cuts	Every beat	Every other beat	Every 3rd beat
CTA Cuts	2+ in last 3s	1 in last 3s	1 hard cut
Text Density	80% of words	45% of words	20% of words
Camera Move Interval	1500 ms	3000 ms	5000 ms
SFX Density	100% (every event)	50% (every other)	0% (no SFX)
Animation Pool	pop, glitch, elastic_scale	slide_up, pop, typewriter	slide_up, typewriter
High-Energy Color	#FF0040 (red)	#FF0040 (red)	#FFFFFF (white)
Breathe Frequency	Rare (intense)	Moderate (strategic)	Frequent (calm)
Candidate Count	4–6	3–4	2–3

Text Placement Rules

Rule	Value	Enforcement
Y-position range	0.0 – 0.4 (top 40%)	Hard clamp in Director + Composition
X-position	0.5 (center) ± 0.15 jitter	Seeded randomness
Scale	1.0 (normal) to 1.3 (high-energy)	Physics engine
Max words on screen	3	Critic layer validation
Min spacing between text events	200 ms	Critic layer validation
Color contrast	> 4.5:1 against video	Accessibility

SFX Cue Categories

Category	Variations	Trigger	duckMusicDb
whoosh_fast	5	Hard cut	-6
whoosh_slow	5	Zoom blur cut	-6
impact_deep	5	Red text (non-CTA)	-9
impact_sharp	5	Red text (CTA)	-9
riser_short	5	Pre-breathe	-3
sub_drop	5	High-energy beat (>0.85)	-9
glitch_digital	5	Glitch animation text	-6
pop_text	5	Pop animation text	-6

The "Breathe Beat" Specification

Trigger Condition	Visual	Audio	Text
Intensity budget exhausted (cumulative energy > 1.0 for > 1.5s over 2s)	Slow zoom > 0.7 for > 1.5s	Music swell (+3 dB)	**NONE**
Low-energy phrase boundary (< 0.35)	Static frame or gentle drift	Ambient music	**NONE**
Post-high-energy sequence	Camera reset to neutral	SFX tail	**NONE**

Critical: The breathe is NOT dead air. It is intense visual focus without text. The viewer should feel the speaker's presence, not a bug.

12. THE ASSET CONTRACT (NEW)

The Problem

The browser composition and the FFmpeg layer do not want the same representation. `file:///` is blocked in Chromium. Public URLs are unavailable to FFmpeg.

The Contract

Each media field in `UnifiedRenderManifest` must specify its representation:

```
interface MediaReference {  
  browserUrl: string; // What R3F/Remotion loads (e.g., '/dev-fixtures/video.mp4')  
  filePath: string;   // What FFmpeg loads (e.g., 'C:/.../public/dev-fixtures/video.mp4')  
  assetId: string;    // Abstract identifier (e.g., 'asset://video_1')  
}
```

Resolution Strategy

Stage	Needs	Resolution
Director	Abstract asset IDs	<code>`asset://video_1`, `asset://music_1`</code>
Asset Resolver	Converts to concrete paths	<code>`browserUrl`</code> for Remotion, <code>`filePath`</code> for FFmpeg
Remotion	Browser-served URLs	Relative paths from <code>`public`</code> directory
FFmpeg	Absolute file paths	<code>`path.resolve()`</code> from project root

Implementation

```
// packages/shared-types/src/asset-resolver.ts  
export interface AssetResolver {  
  resolveBrowser(assetId: string): string;  
  resolveFile(assetId: string): string;  
}  
  
export class LocalAssetResolver implements AssetResolver {  
  private publicDir: string;  
  
  resolveBrowser(assetId: string): string {  
    return assetId.replace('asset://', '/');  
  }  
  
  resolveFile(assetId: string): string {  
    return path.join(this.publicDir, assetId.replace('asset://', ''));  
  }  
}
```

Why This Matters

Without this contract, you get contradictory bugs:

- `file:///` URLs blocked by Chromium
- Public URLs unavailable to FFmpeg
- Schema accepting one representation while runtime expects another

13. THE RENDER ENTRY CONTRACT (NEW)

The Problem

Bundling the entire remotion-app tree to render one Joseph composition is expensive. It imports unrelated preview/editor logic, causing longer bundle times and more incidental failures.

The Contract

Create a Joseph-specific render entry that owns:

- Composition registration for Joseph only
- The public assets it needs
- The exact props schema it expects
- The preview/render parity math for that path only

Implementation

```
// remotion-app/src/joseph-entry.tsx
import { Composition } from 'remotion';
import { JosephEdit } from '../compositions/JosephEdit';

export const JosephEntry = () => {
  <Composition
    id="JosephEdit"
    component={JosephEdit}
    durationInFrames={2700}
    fps={30}
    width={1080}
    height={1920}
    defaultProps={{
      manifest: null,
      frameData: null,
    }}
  />
};
```

Bundle Entry Point

```
// remotion-app/src/joseph-bundle.ts
import { registerRoot } from 'remotion';
import { JosephEntry } from '../joseph-entry';

registerRoot(JosephEntry);
```

Worker Usage

```
// apps/worker/src/index.ts
const serveUrl = await bundle({
  entryPoint: path.resolve(__dirname, '../../../remotion-app/src/joseph-bundle.ts'),
  // NOT the full app root
```

});

Why This Matters

- Shorter bundle time
- Less font/bootstrap noise
- Fewer incidental failures
- Stronger preview/render isolation
- Clean Phase 2 hook (wrap renderFromManifest, not rewrite)

14. THE 2-WEEK SPRINT PLAN v8.1

Week 1: The Render Pipeline

Day	Ticket	Agent	Files	Deliverable	Gate
1	**T1: Bundle Cache**	GPT	`apps/worker/src/index.ts`	`bundle()` runs once, caches results. Second render skips bundle.	
1	**T2: Vertical Resolution**	Opus	`remotion-app/src/composition.ts`	1080s1024pH264H.265Text clamped to y: 0.0–0.4.	
2	**T3: Physics Engine**	GPT	`packages/shared-utils/src/physics.ts`	Physics 200 events × 270 frames in <500ms.	
2	**T4: Font Fallback**	Opus	`remotion-app/src/composition.ts`	500msKineticText.HREF.Texture. Deterministic.	
3	**T5: R3F Integration**	Opus	`remotion-app/src/composition.ts`	Stands/Joints/BEA/Axis from `inputProps`. No real-time physics.	
3	**T6: Render Entry Control**	GPT	`remotion-app/src/joseph-render-entry.ts`	Joseph-type Joseph-typeNo unrelated app code.	
4	**T7: Asset Contract**	GPT	`packages/shared-types/src/asset-contract.ts`	FFmpeg media paths.	
4	**T8: Chrome Timeout**	GPT	`apps/worker/src/index.ts`	5-min timeout. `SIGKILL` on hang.	
5	**T9: Integration Test**	Both	`scripts/test-joseph.ts`	`--quick` passes in <3ms.	Gate 1: Determinism check.

Week 2: The Director + Judgment

	Ticket	Agent	Files	Deliverable	Gate			
	T10: Candidate Generation	Opus	`backend/src/director/joseph-candidate.ts`	Initial 26 candidate Treatment Genomes per profile.				
	T11: Judgment Layer	Opus	`backend/src/director/judgment-layer.ts`	Quality filter: anti-repetition, similarity veto.				
	T12: Replay Ledger	GPT	`backend/src/ledger/replay-queue.ts`	Sequence schema. CRUD operations.				
	T13: Dynamic Boundary Estimation	Opus	`backend/src/director/joseph-dynamic-boundary-estimation.ts`	Plus random beats, not machine.	Gate 2.			
	T14: Sequence Memory	Opus	`backend/src/director/sequence-memory.ts`	State machine: calm building	saturated	recovering.	—	
	T15: Band-Pass Ducking	GPT	`backend/src/audio/mix-and-duck.ts`	Adapt pumping on drum beats.				
	T16: SFX Variations	GPT	`remotion-app/public/sfx/40-director.ts`	40 Directories generated. Director selects seeded variation.				

	Ticket	Agent	Files	Deliverable	Gate			
	**T17: Variation Contract	Opus	`backend/src/director/variation_keys.ts`	Explicitly load instance_id + retry_index handling.				
	**T18: Full Integration	Both	`scripts/test-joseph.ts`	`--full` passes. 3 profiles created. Re-upload variation works.				
	**T19: Evidence Preservation	BBT	`backend/src/ledger/evidence_preservation.ts`	Preserve values, rejections, verdicts.				
	Buffer	Both	—	Bug fixes, edge cases, performance tuning.				
	CI + Docs	GPT	`_github/workflows/`, `FAIRPLAY/README.md`	Deployment CI + Docs documented.				

Parallelization Schedule

```

Day 1:  T1 (Bundle)      || T2 (Vertical)
Day 2:  T3 (Physics)    || T4 (Font)
Day 3:  T5 (R3F)        || T6 (Render Entry)
Day 4:  T7 (Asset)      || T8 (Timeout)
Day 5:  T9 (Integration Test)
Day 6:  T10 (Candidates)
Day 7:  T11 (Judgment)   || T12 (Ledger)
Day 8:  T13 (Boundaries)
Day 9:  T14 (Seq Memory) || T15 (Ducking)
Day 10: T16 (SFX)        || T17 (Variation)
Day 11: T18 (Full Integration)
Day 12: T19 (Evidence)
Day 13: Buffer
Day 14: CI + Docs

```

Cut Priority (If Behind Schedule)

1. **Cut first:** SFX variations → use 8 placeholder MP3s (1 variation per cue).
2. **Cut second:** Evidence Preservation → basic logging only.
3. **Cut third:** Font subsetting → load 5 fonts statically.
4. **Cut fourth:** Anti-repetition → basic "no same effect within 10s" rule.
5. **NEVER CUT:** Bundle cache, determinism, vertical resolution, physics engine, candidate generation, Judgment Layer.

15. THE BUILD ORDER (EXACT FILES, EXACT CODE)

[Note: Steps 1–12 from v8 are preserved. Steps 6, 7, and 8 are expanded with new contracts.]

Step 1: Bundle Cache (Day 1)

File: apps/worker/src/index.ts

```

let cachedBundle: { key: string; serveUrl: string } | null = null;

function getBundleCacheKey(): string {
  const files = globSync(

```

```

path.resolve(__dirname, '../../../remotion-app/src/**/*.ts,tsx}')
);
const hash = createHash('sha256');
files.sort().forEach((f) => hash.update(fs.readFileSync(f)));
return hash.digest('hex').slice(0, 16);
}

export async function renderFromManifest(manifest: UnifiedRenderManifest): Promise<string> {
const key = getBundleCacheKey();

if (!cachedBundle || cachedBundle.key !== key) {
const serveUrl = await bundle({
entryPoint: path.resolve(__dirname, '../../../remotion-app/src/joseph-bundle.ts'),
webpackOverride: (config) => {
config.resolve ||= {};
config.resolve.alias ||= {};
config.resolve.alias['@prometheus/shared-types'] = path.resolve(
__dirname, '../../../packages/shared-types/src'
);
return config;
},
});
cachedBundle = { key, serveUrl };
}

const { serveUrl } = cachedBundle;
const composition = await selectComposition({ serveUrl, id: 'JosephEdit' });

const outputPath = path.join('output', `${manifest.jobId}.mp4`);

const timeout = setTimeout(() => {
throw new Error('Render timeout after 5 minutes');
}, 5 * 60 * 1000);

await renderMedia({
composition,
serveUrl,
codec: 'H264',
crf: 18,
outputLocation: outputPath,
inputProps: { manifest, frameData: computePhysics(manifest) },
gl: 'angle',
});

clearTimeout(timeout);
return outputPath;
}

```

Step 2: Vertical Resolution (Day 1)

File: remotion-app/src/compositions/JosephEdit.tsx

```

const { width, height } = useVideoConfig(); // 1080, 1920
// Text: y position clamped to 0.0-0.4 (top 40%)
// Video: center crop to 9:16 aspect ratio

```

Step 3: Physics Engine (Day 2)

File: packages/shared-utils/src/physics-engine.ts

```

export function computePhysics(manifest: UnifiedRenderManifest): FrameData {
const { textOverlays, durationFrames, seed } = manifest;
const rng = seededRandom(seed);
const maxWords = textOverlays.length;

```

```

const transforms = new Float32Array(maxWords * durationFrames * 4);
const active = new Uint8Array(maxWords * durationFrames);

transforms.fill(0);
active.fill(0);

for (let w = 0; w < maxWords; w++) {
  const overlay = textOverlays[w];
  const startFrame = overlay.startFrame;
  const endFrame = Math.min(overlay.endFrame, durationFrames);

  for (let f = startFrame; f < endFrame; f++) {
    const idx = (w * durationFrames + f) * 4;
    const progress = (f - startFrame) / (endFrame - startFrame);

    const t = progress;
    const overshoot = Math.exp(-t * 5) * Math.sin(t * 10);
    const settle = 1 - Math.exp(-t * 3);

    transforms[idx] = 0.5 + (rng() - 0.5) * 0.3;
    transforms[idx + 1] = overlay.position.y + overshoot * 0.05;
    transforms[idx + 2] = overlay.scale * (1 + overshoot * 0.2);
    transforms[idx + 3] = overshoot * 0.1;
    active[w * durationFrames + f] = 1;
  }
}

return { transforms, active, durationFrames };
}

```

Step 4: Font Fallback (Day 2)

File: remotion-app/src/compositions/KineticText.tsx

```

function createTextTexture(text: string, fontFamily: string, fontSize: number, color: string):
THREE.CanvasTexture {
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d')!;
  ctx.font = `bold ${fontSize}px "${fontFamily}", sans-serif`;
  const metrics = ctx.measureText(text);
  canvas.width = Math.ceil(metrics.width);
  canvas.height = Math.ceil(fontSize * 1.5);
  ctx.font = `bold ${fontSize}px "${fontFamily}", sans-serif`;
  ctx.fillStyle = color;
  ctx.textBaseline = 'top';
  ctx.fillText(text, 0, 0);
  return new THREE.CanvasTexture(canvas);
}

```

Step 5: R3F Integration (Day 3)

File: remotion-app/src/compositions/JosephEdit.tsx

```

const { frameData } = useVideoConfig().inputProps as { frameData: FrameData };
const frame = useCurrentFrame();

const textElements = textOverlays.map((overlay, index) => {
  const f = Math.min(frame, frameData.durationFrames - 1);
  const activeIdx = index * frameData.durationFrames + f;
  if (frameData.active[activeIdx] === 0) return null;

  const tIdx = activeIdx * 4;
  return (
    <Text

```

```

position=[frameData.transforms[tIdx], frameData.transforms[tIdx + 1], 0]]
fontSize={overlay.fontSize * frameData.transforms[tIdx + 2]}
rotation=[0, 0, frameData.transforms[tIdx + 3]]
color={overlay.color}
<
{overlay.text}
</Text>
);
});

```

Step 6: Render Entry Contract (Day 3)

Files: remotion-app/src/joseph-bundle.ts, remotion-app/src/joseph-entry.tsx

```

// joseph-entry.tsx
import { Composition } from 'remotion';
import { JosephEdit } from '../compositions/JosephEdit';

export const JosephEntry = () => (
  <Composition
    id="JosephEdit"
    component={JosephEdit}
    durationInFrames={2700}
    fps={30}
    width={1080}
    height={1920}
    defaultProps={{ manifest: null, frameData: null }}
  />
);

// joseph-bundle.ts
import { registerRoot } from 'remotion';
import { JosephEntry } from './joseph-entry';
registerRoot(JosephEntry);

```

Step 7: Asset Contract (Day 4)

File: packages/shared-types/src/asset-resolver.ts

```

export interface MediaReference {
  assetId: string;
  browserUrl: string;
  filePath: string;
}

export interface AssetResolver {
  resolveBrowser(assetId: string): string;
  resolveFile(assetId: string): string;
}

export class LocalAssetResolver implements AssetResolver {
  constructor(private publicDir: string) {}

  resolveBrowser(assetId: string): string {
    return assetId.replace('asset://', '/');
  }

  resolveFile(assetId: string): string {
    return path.join(this.publicDir, assetId.replace('asset://', ''));
  }
}

```

Step 8: Chrome Timeout (Day 4)

File: apps/worker/src/index.ts

```
const RENDER_TIMEOUT_MS = 5 * 60 * 1000;

export async function renderFromManifest(manifest: UnifiedRenderManifest): Promise<string> {
  // ... setup ...

  return new Promise((resolve, reject) => {
    const timeout = setTimeout(() => {
      reject(new Error(`Render timeout after ${RENDER_TIMEOUT_MS}ms`));
    }, RENDER_TIMEOUT_MS);

    renderMedia({ /* ... */ })
      .then((result) => { clearTimeout(timeout); resolve(result); })
      .catch((err) => { clearTimeout(timeout); reject(err); });
  });

  process.on('exit', () => {
    try {
      const { execSync } = require('child_process');
      execSync('taskkill /F /IM chrome.exe 2>nul || true');
    } catch {}
  });
}
```

Step 9: Candidate Generation (Day 6)

File: backend/src/director/joseph-director.ts

```
export function generateCandidateGenomes(
  input: DirectorInput,
  count: number
): UnifiedRenderManifest[] {
  const candidates: UnifiedRenderManifest[] = [];

  for (let i = 0; i < count; i++) {
    const seed = input.seed + i * 7919; // Deterministic offset
    candidates.push(generateJosephManifest({ ...input, seed }));
  }

  return candidates;
}
```

Step 10: Judgment Layer (Day 7)

File: backend/src/director/judgment-layer.ts

```
export class JudgmentLayer {
  constructor(private ledger: ReplayLedger) {}

  async judgeCandidates(
    candidates: UnifiedRenderManifest[],
    variationKey: VariationKey
  ): Promise<{ selected: UnifiedRenderManifest; rejected: UnifiedRenderManifest[] }> {
    const qualified = candidates.filter((c) => this.meetsQualityFloor(c));
    const novel = await this.filterBySimilarity(qualified, variationKey);

    if (novel.length === 0) {
      throw new Error('No novel candidates passed judgment');
    }
  }
}
```

```

const selected = this.selectByVariationKey(novel, variationKey);
const rejected = candidates.filter((c) => c !== selected);

return { selected, rejected };
}

private meetsQualityFloor(manifest: UnifiedRenderManifest): boolean {
// Min cuts, max text overlap, readability checks
return true; // Implement checks
}

private async filterBySimilarity(
candidates: UnifiedRenderManifest[],
variationKey: VariationKey
): Promise<UnifiedRenderManifest[]> {
const prior = await this.ledger.getBySource(variationKey.sourceFingerprint);
// Remove candidates with similarity > 0.85 to prior outputs
return candidates;
}

private selectByVariationKey(
candidates: UnifiedRenderManifest[],
variationKey: VariationKey
): UnifiedRenderManifest {
const rng = seededRandom(variationKey.retryIndex);
const index = Math.floor(rng() * candidates.length);
return candidates[index];
}
}

```

Step 11: Replay Ledger (Day 7)

File: backend/src/ledger/replay-ledger.ts

```

import Database from 'better-sqlite3';

export class ReplayLedger {
private db: Database.Database;

constructor(dbPath: string = './.prometheus/replay-ledger.sqlite') {
this.db = new Database(dbPath);
this.init();
}

private init() {
this.db.exec(`
CREATE TABLE IF NOT EXISTS replay_ledger (
id TEXT PRIMARY KEY,
source_fingerprint TEXT NOT NULL,
prompt_fingerprint TEXT,
upload_instance_id TEXT NOT NULL,
retry_index INTEGER DEFAULT 0,
profile TEXT NOT NULL,
chosen_genome TEXT NOT NULL,
rejected_genomes TEXT,
planner_audit TEXT,
similarity_hash TEXT,
quality_score REAL,
failure_tags TEXT,
created_at TEXT
);
CREATE INDEX IF NOT EXISTS idx_source ON replay_ledger(source_fingerprint);
CREATE INDEX IF NOT EXISTS idx_prompt ON replay_ledger(prompt_fingerprint);
`);
}
}

```



```

insert(entry: LedgerEntry): void {
  const stmt = this.db.prepare(`
  INSERT INTO replay_ledger VALUES (
    @id, @source_fingerprint, @prompt_fingerprint, @upload_instance_id,
    @retry_index, @profile, @chosen_genome, @rejected_genomes,
    @planner_audit, @similarity_hash, @quality_score, @failure_tags, @created_at
  )
  `);
  stmt.run(entry);
}

getSource(sourceFingerprint: string): LedgerEntry[] {
  return this.db.prepare(
    'SELECT * FROM replay_ledger WHERE source_fingerprint = ? ORDER BY created_at DESC'
  ).all(sourceFingerprint);
}

```

Step 12: Dynamic Boundaries (Day 8)

File: backend/src/director/joseph-director.ts

```

function findCutPoints(phrases: Phrase[], beats: number[], onsets: number[], durationMs: number):
number[] {
  const cuts: number[] = [];
  const allSyncPoints = [...beats, ...onsets].sort((a, b) => a - b);

  const hookCuts = allSyncPoints.filter((p) => p <= 3000).slice(0, 5);
  cuts.push(...hookCuts);

  for (const phrase of phrases.slice(0, -1)) {
    const boundary = phrase.endMs;
    const nearestBeat = findNearest(boundary, allSyncPoints);
    if (Math.abs(nearestBeat - boundary) < 100) {
      cuts.push(nearestBeat);
    }
  }

  const ctaStart = durationMs - 3000;
  const ctaCuts = allSyncPoints.filter((p) => p >= ctaStart);
  cuts.push(...ctaCuts);

  return [...new Set(cuts)].sort((a, b) => a - b);
}

```

Step 13: Sequence Memory State Machine (Day 9)

File: backend/src/director/sequence-memory.ts

```

export type SequenceState = 'calm' | 'building' | 'saturated' | 'recovering';

export interface SequenceMemory {
  state: SequenceState;
  intensityBudget: number;
  cumulativeEnergy: number[];
  lastBreatheFrame: number;
  lastEffectFrame: Map<string, number>;
}

export function createMemory(): SequenceMemory {
  return {
    state: 'calm',
    intensityBudget: 1.0,
    cumulativeEnergy: [],
  };
}

```

```

lastBreatheFrame: 0,
lastEffectFrame: new Map(),
};
}

export function updateMemory(
  memory: SequenceMemory,
  energy: number,
  frame: number,
  isCTA: boolean
): SequenceMemory {
  memory.cumulativeEnergy.push(energy);
  const windowSize = 20 * 30; // 20 seconds
  const recent = memory.cumulativeEnergy.slice(-windowSize);
  const avgEnergy = recent.reduce((a, b) => a + b, 0) / recent.length;

  // State transitions
  if (isCTA) {
    memory.state = 'building'; // CTA always builds
  } else if (avgEnergy > 0.7 && memory.state !== 'saturated') {
    memory.state = 'building';
    memory.intensityBudget -= 0.05;
  } else if (memory.intensityBudget <= 0) {
    memory.state = 'saturated';
  } else if (memory.state === 'saturated' && avgEnergy < 0.4) {
    memory.state = 'recovering';
    memory.intensityBudget = Math.min(1.0, memory.intensityBudget + 0.2);
  } else if (memory.state === 'recovering' && avgEnergy < 0.3) {
    memory.state = 'calm';
  }

  return memory;
}

export function shouldBreathe(memory: SequenceMemory, frame: number): boolean {
  if (memory.state === 'saturated' && frame - memory.lastBreatheFrame > 10 * 30) {
    memory.lastBreatheFrame = frame;
    memory.state = 'recovering';
    memory.intensityBudget = 1.0;
    return true;
  }
  return false;
}

export function canUseEffect(memory: SequenceMemory, effect: string, frame: number): boolean {
  const last = memory.lastEffectFrame.get(effect);
  if (last !== undefined && frame - last < 15 * 30) {
    return false;
  }
  return true;
}

export function useEffect(memory: SequenceMemory, effect: string, frame: number): void {
  memory.lastEffectFrame.set(effect, frame);
  memory.intensityBudget -= 0.15;
}

```

Step 14: Band-Pass Ducking (Day 9)

File: backend/src/audio/mix-audio.ts

```

ffmpeg -y -i mixed.mp3 -filter_complex "
[0:a]asplit=2[sc][mix];
[sc]bandpass=f=1000:width_type=h:w=1200[sidechain];
[sidechain]agate=threshold=-25dB:ratio=4:attack=50:release=200[gate];
[mix]sidechaincompress=level_in=1:level_out=1:ratio=4:threshold=0.05:attack=50:release=200:detection

```

```
=peak:mix=1:sidechain=1[ducked];
[gate][ducked]amix=inputs=2:duration=first:dropout_transition=2
" -c:a aac -b:a 192k -ar 48000 -target_loudness -14 output.mp3
```

Step 15: SFX Variations (Day 10)

Directory: remotion-app/public/sfx/

```
for cue in whoosh_fast whoosh_slow impact_deep impact_sharp riser_short sub_drop glitch_digital
pop_text; do
for i in 1 2 3 4 5; do
ffmpeg -f lavfi -i anullsrc=r=44100:cl=mono -t 0.5 -acodec libmp3lame -q:a 9 "remotion-
app/public/sfx/${cue}_${i}.mp3"
done
done
```

Step 16: Variation Contract (Day 10)

File: backend/src/director/variation-key.ts

```
export interface VariationKey {
  sourceFingerprint: string;
  promptFingerprint: string;
  uploadInstanceId: string;
  retryIndex: number;
}

export function generateVariationKey(
  sourceVideoPath: string,
  prompt: string,
  uploadInstanceId: string = randomUUID(),
  retryIndex: number = 0
): VariationKey {
  const sourceFingerprint = createHash('sha256')
    .update(fs.readFileSync(sourceVideoPath))
    .digest('hex');
  const promptFingerprint = createHash('sha256')
    .update(prompt)
    .digest('hex');

  return { sourceFingerprint, promptFingerprint, uploadInstanceId, retryIndex };
}
```

Step 17: Integration (Day 11)

File: scripts/test-joseph.ts

```
// Run 3 profiles
// Verify distinct outputs
// Verify determinism (same key = same SHA256)
// Verify variation (different uploadInstanceId = different output)
// Verify size > 100KB
// Verify vertical 1080x1920
```

Step 18: Evidence Preservation (Day 12)

File: backend/src/ledger/evidence-preservation.ts

```
export function preserveEvidence(
```

```

variationKey: VariationKey,
candidates: UnifiedRenderManifest[],
selected: UnifiedRenderManifest,
rejected: UnifiedRenderManifest[],
verdict: JudgmentVerdict
): void {
  const evidence = {
    variationKey,
    candidateCount: candidates.length,
    selectedGenome: selected,
    rejectedGenomes: rejected,
    verdict,
    timestamp: new Date().toISOString(),
  };

  fs.appendFileSync(
    'evidence.log',
    JSON.stringify(evidence) + '\n'
  );
}

```

Step 19: CI + Documentation (Day 14)

Files: .github/workflows/determinism.yml, FAILURE_TAXONOMY.md

```

name: Prometheus CI
on: [push, pull_request]

jobs:
  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      with: { node-version: '20' }
      - run: npm ci
      - run: npm run build --workspace=packages/shared-types
      - run: npm run typecheck --workspace=packages/shared-types
      - run: npm run typecheck --workspace=backend
      - run: npm run typecheck --workspace=apps/worker
      - run: npm run typecheck --workspace=remotion-app

  test-backend:
    runs-on: ubuntu-latest
    needs: typecheck
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      with: { node-version: '20' }
      - run: npm ci
      - run: npm run test --workspace=backend

  determinism:
    runs-on: ubuntu-latest
    needs: typecheck
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      with: { node-version: '20' }
      - run: npm ci
      - run: npm run build --workspace=packages/shared-types
      - run: npx tsx scripts/test-joseph.ts --quick --hash ci-test-123
      - name: Verify no zombies
      run: |
        sleep 5

```

```

if pgrep -f chrome; then echo "Zombie Chrome!"; exit 1; fi
- name: Verify determinism (same key)
run: |
HASH1=$(sha256sum output/joseph/edit_0_joseph_aggressive.mp4 | cut -d' ' -f1)
# Render again with same key
npx tsx scripts/test-joseph.ts --quick --hash ci-test-123
HASH2=$(sha256sum output/joseph/edit_0_joseph_aggressive.mp4 | cut -d' ' -f1)
if [ "$HASH1" != "$HASH2" ]; then echo "Determinism failed!"; exit 1; fi
- name: Verify variation (different key)
run: |
# Render with different upload_instance_id
npx tsx scripts/test-joseph.ts --quick --hash ci-test-456
HASH3=$(sha256sum output/joseph/edit_0_joseph_aggressive.mp4 | cut -d' ' -f1)
if [ "$HASH1" = "$HASH3" ]; then echo "Variation failed!"; exit 1; fi

```

16. PERFORMANCE & SCALING MATH

Render Time Equation (Sequential)

$$T_{\text{total}} = T_{\text{bundle}} + T_{\text{render}} + T_{\text{audio}} + T_{\text{mux}} + T_{\text{judgment}}$$

$T_{\text{bundle}} = 42\text{s}$ (one-time with cache)
 $T_{\text{render}} = \text{frames} / (\text{fps} \times \text{render_speed_factor})$
 $T_{\text{audio}} = 5\text{--}10\text{s}$ (FFmpeg mixing)
 $T_{\text{mux}} = 2\text{--}5\text{s}$ (FFmpeg concat video + audio)
 $T_{\text{judgment}} = 50\text{--}100\text{ms}$ (candidate generation + selection)

Software Rendering (`swangle`)

Parameter	Value
render_speed_factor	0.3 (3 sec video per 10 sec wall time)
90 sec video	2700 frames / (30 × 0.3) = 300s = 5 min
+ bundle	5.7 min
+ audio	5.8 min
+ mux	5.9 min
+ judgment	5.9 min
Total	**~6 minutes**

GPU Rendering (`angle`)

Parameter	Value
render_speed_factor	2.0 (2× real-time)
90 sec video	2700 frames / (30 × 2.0) = 45s
+ bundle	87s

Parameter	Value
+ audio	97s
+ mux	102s
+ judgment	102s
Total	**~1.7 minutes**

Physics Pre-Computation

Operation	Time
300 words × 2700 frames × 4 floats	~30ms
Array allocation	~5ms
Total	**~35ms**

Candidate Generation

Operation	Time
6 candidates × full manifest generation	~20ms
Judgment Layer similarity check (SQLite)	~10ms
Total	**~30ms**

Memory Budget (Chrome)

Component	Size
FrameData (Float32Array)	~13 MB
Manifest JSON	~50 KB
Video texture (1 frame)	~8 MB
Text textures (5 fonts, subsetting)	~2 MB
Total	**~23 MB**
Chrome headless overhead	~500 MB
Grand Total	**~523 MB**

Cost Math (Phase 2 Cloud)

Instance	GPU	RAM	Cost/Hour	90-sec Render	Cost/Video
g4dn.xlarge	T4	16 GB	\$0.526	~1.7 min	\$0.015

Instance	GPU	RAM	Cost/Hour	90-sec Render	Cost/Video
g4dn.2xlarge	T4	32 GB	\$1.052	~1.7 min (2 concurrent)	\$0.015
g5.xlarge	A10G	16 GB	\$1.006	~1.0 min	\$0.017

17. THE RISK REGISTER v8.1

#	Risk	Likelihood	Impact	Mitigation	Owner
1	`angle` fails on user's machine	High	Existential	Fallback to `swangle`. 6-min render time acceptable for MVP.	Infrastructure
2	DOM canvas text too slow for preview	Medium	Systemic	Preview is still-frame or 500px resolution is what matters.	DevOps
3	Band-pass false positives	High	Isolated	Document in `FAILURE_TAXONOMY.md`.	Physics
4	Agent rewrites R3F to DOM	High	Existential	ESLint + human PR review	DevOps
5	Chrome zombies after timeout	Medium	Systemic	`SIGKILL` in timeout handler	Worker
6	Physics computation > 1s	Low	Systemic	Pre-allocated arrays.	Physics
7	2-week sprint impossible	Medium	Existential	Cut SFX variations, sentiment projection subsetting.	Product
8	Troika SDF fails	Medium	Isolated	DOM canvas fallback is possible	COMPARISON
9	Vertical crop cuts off speaker	Low	Isolated	Center crop. Test with sample videos	Optimization
10	Bundle cache invalidation fails	Low	Systemic	SHA256 of src dir.	Worker
11	Variation feels fake	Medium	Existential	Candidate set + Replay Ledger to play key.	Judgment
12	Governor ambiguity	Medium	Existential	Named Planner/Judgment Selection seams.	Architecture
13	Prompt breaks infrastructure	High	Existential	Prompt Governance Rules	Architecture
14	Evidence lost for future IR	Medium	Systemic	Evidence Preservation Path	Architecture
15	Schema/runtime drift	Medium	Systemic	CI gate for source/dist sync	DevOps
16	Placement failures unmeasured	Medium	Isolated	Placement-failure tags in Director Surface.	Director
17	Candidate generation tool slow	Low	Systemic	Bounded set (2-6 candidates)	Director
18	Replay Ledger corruption	Low	Systemic	SQLite with WAL mode. Backup strategy.	Backend

18. THE AGENT PROTOCOL

Rule 1: Ticket Boundaries

Each agent receives exactly one ticket. The prompt includes:

- The exact file(s) to edit

- The exact function signature to implement
- The exact test that must pass
- The list of files that are FORBIDDEN to touch

Rule 2: No Architecture Changes

Agents may not:

- Change UnifiedRenderManifest schema
- Change AGENTS.md or CONTEXT.md
- Add new dependencies without approval
- Switch from R3F to DOM
- Remove determinism constraints
- Introduce Math.random, Date.now(), performance.now(), setTimeout, setInterval, or GSAP in render path
- Change the Governor (Judgment Layer) authority
- Bypass Prompt Governance Rules

Rule 3: Fallback First

If an agent hits a blocker, the prompt must include:

- **Primary approach** (what to try first)
- **Fallback approach** (what to do if primary fails)
- Explicit instruction: *"Do not spend more than 2 hours on the primary. If it fails, switch to fallback and report."*

Rule 4: Determinism Audit

Every PR must include:

- The exact command used to verify determinism
- The SHA256 output (or "N/A" if non-visual change)

Rule 5: Human Gate

Only the human architect may merge PRs that touch:

- UnifiedRenderManifest
- remotion-app/src/compositions/JosephEdit.tsx
- apps/worker/src/index.ts (render path)
- packages/shared-types/src/ (schema)
- backend/src/director/judgment-layer.ts (governor)
- backend/src/director/variation-key.ts (variation contract)

- backend/src/ledger/replay-ledger.ts (memory)

Rule 6: Build Governor Protocol

- **Schema changes:** Human approval required
- **Render entry changes:** Human approval required
- **Director heuristics changes:** Allowed with regression tests
- **SFX library additions:** Allowed if cue contract unchanged
- **Asset-serving changes:** Require explicit review (affects browser/FFmpeg seam)
- **Judgment Layer changes:** Human approval required
- **Variation contract changes:** Human approval required

19. THE GO/NO-GO GATES v8.1

Gate	Day	Check	Pass Criteria	Blocker Action
0	0	Environment	`npm run typecheck` passes in all workspaces, no project has any passes in backend	
1	1	Bundle Cache	Second render reuses bundle (no 42s delay) key logic.	
2	2	Physics	300 words × 2700 frames in < 1.5s and optimize array allocation.	
3	3	Determinism (Same Key)	Same seed + same uploadInstanceId → Same SHA256.	
3b	3	Variation (Different Key)	Different uploadInstanceId → Different SHA256 key handling.	
4	5	Quick Render	`--quick` completes in < 3 min	Profile render path.
5	7	Director	Cuts on beats, not mid-word.	Fix cut planning logic.
6	10	Full Render	`--full` completes in < 20 min (CPU/GPU/IO/chain/triangle).	
7	12	Integration	3 distinct profiles, all > 100KB	Begin Cardano 1.12 differentiation.
8	14	CI	GitHub Actions determinism + Variation tests pass	No zombie Chrome.

20. THE PHASE 2 HOOK

When we add distributed infrastructure in Phase 2, we do NOT rewrite the render logic. We wrap it:

```
// Phase 2: BullMQ worker
import { Worker } from 'bullmq';
import { renderFromManifest } from './render'; // Same function from MVP

new Worker('render', async (job) => {
  const { manifest, variationKey } = job.data;
  // This is the exact same function from MVP
  const outputPath = await renderFromManifest(manifest);
```

```
await uploadToS3(outputPath);
await notifyWebhook(job.data.webhookUrl, { status: 'done', url: s3Url });
});
```

Phase 2 Additions (Not Replacements)

MVP (Phase 1)	Phase 2 Addition
Sequential render	Parallel chunking (wrap `renderFromManifest`)
Local CLI	BullMQ + Redis queue
Single machine	AWS g4dn worker farm
No monitor	Monitor process with TCP heartbeat
No stitch	FFmpeg concat (only needed for chunking)
Band-pass ducking	Demucs source separation
Upper-middle heuristic	Mediapipe face detection
DOM canvas text	Troika SDF (if proven deterministic)
90-second cap	5-minute cap (with scene detection)
Rule-based Director	GAIL discriminator (after 100+ annotations)
Local Replay Ledger	Cloud Pattern Memory

21. THE DETERMINISM CONTRACT

Forbidden APIs in Render Path

Any code inside remotion-app/src/compositions/** or called during renderMedia() must NOT use:

API	Why Forbidden
`Math.random()`	Non-deterministic sequence
`Date.now()`	Time-dependent
`performance.now()`	Time-dependent
`requestAnimationFrame()`	Frame rate dependent
`setInterval()`	Time-dependent
`setTimeout()`	Time-dependent
`GSAP`	Internal clock, non-deterministic easing
`crypto.getRandomValues()`	Non-deterministic
`new Date()`	Time-dependent

API	Why Forbidden
<code>`Math.sin()` / <code>`Math.cos()`</code> in shaders</code>	Floating-point differences across GPU

Allowed APIs

API	Usage
<code>`seededRandom(seed)`</code>	From <code>`packages/shared-types`</code>
<code>`useCurrentFrame()`</code>	From Remotion. The only time source.
<code>`useVideoConfig()`</code>	From Remotion. Static config.
Pure functions of <code>`(frame, seed, manifest)`</code>	Deterministic by definition
<code>`Float32Array`</code> / <code>`Uint8Array`</code>	Typed arrays for zero-GC data transfer

CI Enforcement

```
# ESLint rule
"no-restricted-globals": ["error", "Math", "Date", "performance"]

# Determinism test
npm run test:determinism
# Generates manifest → renders twice with same key → asserts SHA256 match

# Variation test
npm run test:variation
# Generates manifest → renders with different keys → asserts SHA256 DIFFERENT
```

Schema as Contract (Not Governor)

- All changes to UnifiedRenderManifest require PR review
- Zod validation at worker entry: rejects unknown fields
- Version field in manifest: version: '2.0'
- Schema enforces shape. It does NOT enforce quality, novelty, or taste.

22. THE FFMPEG SPECIFICATION

Render Output (Remotion → Silent MP4)

```
ffmpeg -y -f image2pipe -framerate 30 -i - -c:v libx264 -preset medium -crf 18 -r 30 -vsync cfr
-pix_fmt yuv420p -color_primaries bt709 -color_trc bt709 -colorspace bt709 -movflags +faststart
silent.mp4
```

Audio Mix (FFmpeg ``filter_complex``)

```
ffmpeg -y -i mixed_audio.mp3 -filter_complex "
```

```
[0:a]asplit=2[sc][mix];
[sc]bandpass=f=1000:width_type=h:w=1200[sidechain];
[sidechain]agate=threshold=-25dB:ratio=4:attack=50:release=200[gate];
[mix]sidechaincompress=level_in=1:level_out=1:ratio=4:threshold=0.05:attack=50:release=200:detection
=peak:mix=1:sidechain=1[ducked];
[gate][ducked]amix=inputs=2:duration=first:dropout_transition=2
" -c:a aac -b:a 192k -ar 48000 -target_loudness -14 audio.mp3
```

Final Mux (Video + Audio)

```
ffmpeg -y -i silent.mp4 -i audio.mp3 -c:v copy -c:a copy -movflags +faststart final.mp4
```

Vertical Crop (if source is horizontal)

```
ffmpeg -y -i source.mp4 -vf "crop=ih*9/16:ih,scale=1080:1920" -c:v libx264 -preset medium -crf
18 -r 30 -c:a copy vertical.mp4
```

23. THE PHYSICS ENGINE SPECIFICATION

Input

```
interface PhysicsInput {
  textOverlays: TextOverlay[];
  durationFrames: number;
  seed: number;
  profile: 'joseph_aggressive' | 'joseph_cinematic' | 'joseph_minimal';
}
```

Output

```
interface FrameData {
  transforms: Float32Array; // [x, y, scale, rotation] per word per frame
  active: Uint8Array; // 1 if visible
  durationFrames: number;
}
```

Memory Layout

```
For word index w and frame index f:
transformIndex = (w * durationFrames + f) * 4
x = transforms[transformIndex + 0]
y = transforms[transformIndex + 1]
scale = transforms[transformIndex + 2]
rotation = transforms[transformIndex + 3]

activeIndex = w * durationFrames + f
isVisible = active[activeIndex] === 1
```

Physics Parameters by Profile

Parameter	Aggressive	Cinematic	Minimal
Spring tension	15	8	4
Spring friction	0.3	0.6	0.9
Overshoot amplitude	0.15	0.08	0.03
Position jitter	±0.15	±0.10	±0.05
Scale bounce	0.30	0.15	0.05
Rotation wobble	0.15 rad	0.08 rad	0.00 rad
Entry duration	12 frames	18 frames	24 frames

Spring Equation (Fixed-Point)

```

progress = (frame - startFrame) / durationFrames
overshoot = exp(-progress * tension) * sin(progress * 10)
settle = 1 - exp(-progress * friction * 5)

x = 0.5 + (seededRandom() - 0.5) * jitter
y = baseY + overshoot * overshootAmplitude
scale = baseScale * (1 + overshoot * scaleBounce)
rotation = overshoot * rotationWobble

```

All values are pre-computed in Node.js and stored in Float32Array.

24. THE FONT PIPELINE SPECIFICATION

Primary: DOM Canvas Fallback

Why: CanvasRenderingContext2D in headless Chrome uses Skia (Chromium's graphics engine). This is identical across Windows, macOS, and Linux. It **reduces** cross-platform font metric divergence compared to OS-native engines (FreeType vs. DirectWrite). It does not **absolutely bypass** OS engines, but it makes the divergence manageable.

Process:

1. Load WOFF2 font via @font-face CSS in Remotion preview
2. In render: create <canvas>, draw text with ctx.fillText()
3. Extract CanvasTexture for Three.js
4. Dispose canvas after texture creation

Memory: 5 fonts × ~50KB subset = 250KB. Trivial.

Stretch: `opentype.js` + Troika

If **DOM canvas** is **too slow**: Use `opentype.js` to parse WOFF2 glyph paths in Node.js, bypassing OS font measurement entirely.

Fallback Chain:

1. Try DOM canvas (fast, deterministic enough)
2. If frame drops in preview → try `opentype.js`
3. If Troika fails → use DOM canvas for both preview and render

Font Governor

Rule	Value
Max fonts per render	5
Selection method	Seeded random from 577 library
Subsetting	Only glyphs for words in transcript
Format	WOFF2
Loading	Static <code>@font-face</code> in CSS

25. THE AUDIO MIXING SPECIFICATION

Input

- Mixed audio file (music + voice)
- SFX cue list (triggerMs, cue file, volume, duckMusicDb)

Output

- Mixed MP3 with voice ducking and SFX

Pipeline

```
mixed_audio.mp3
  [Band-Pass Sidechain] → voice gate
  [Band-Pass Sidechain] triggers music ducking
  [SFX Layer] → adelay per cue + volume adjustment
  [SFX Layer] mixed with amix
  [Loudnorm] → -14 LUFS target
```

Parameters

Parameter	Value	Reason
Band-pass center	1000 Hz	Center of vocal range
Band-pass width	1200 Hz	Covers 300–3000 Hz
Gate threshold	-25 dB	Only strong voice triggers
Gate ratio	4:1	Aggressive isolation
Compressor threshold	0.05 (linear)	Duck music when gate opens
Compressor ratio	4:1	Strong ducking
Compressor attack	50 ms	Fast response to voice
Compressor release	200 ms	Natural fade back
Music level (ducked)	-18 dB	Audible but subordinate
SFX level	-10 to -12 dB	Present but not overwhelming
Target LUFS	-14	Platform standard

Known Limitation (FT-005)

Band-pass sidechain may false-positive on synth leads or guitars in the 300–3000 Hz range. This is acceptable for MVP. Documented in FAILURE_TAXONOMY.md. Phase 2 replaces with Demucs source separation.

26. THE DIRECTOR RULES & HEURISTICS

The Three Profiles

Attribute	Joseph Aggressive	Joseph Cinematic	Joseph Minimal
Hook cuts	5+ in first 3s	3–4 in first 3s	2–3 in first 3s
Body cuts	Every beat	Every other beat	Every 3rd beat
CTA cuts	2+ in last 3s	1 in last 3s	1 hard cut
Text coverage	80% of words	45% of words	20% of words
Text animation	pop, glitch, elastic_scale	slide_up, pop, typewriter	slide_up, typewriter
High-energy color	#FF0040 (red)	#FF0040 (red)	#FFFFFF (white)
Camera moves	push_in every 1.5s	push_in every 3s	push_in every 5s
Dutch/shake	Frequent on beats	Occasional on drops	Never
SFX density	100% (every event)	50% (every other)	0%
Breathe frequency	Rare (max intensity)	Moderate (strategic)	Frequent (calm)
Candidate count	4–6	3–4	2–3

Cut Planning Rules

1. **Hook Rule:** First 3 seconds must have ≥ 2 cuts (all profiles).
2. **Beat Rule:** Every cut must land on a beat or onset ($\pm 100\text{ms}$ tolerance).
3. **Phrase Rule:** Never cut mid-word. Cut at phrase boundaries or silence $> 300\text{ms}$.
4. **CTA Rule:** Last 3 seconds must have ≥ 1 push_in camera move.
5. **Energy Rule:** High-energy sections (> 0.7) get zoom_blur transitions. Low-energy (< 0.3) get hard cuts.

Text Selection Rules

1. **Thesis Words:** Words ≥ 5 letters, not stop words, confidence > 0.9 OR energy > 0.7 .
2. **High-Energy Words:** Thesis words with energy ≥ 0.7 AND confidence ≥ 0.9 . Colored red.
3. **Coverage:** Only thesis words are guaranteed. Other words appear based on profile.textCoverage probability.
4. **CTA Words:** Last 2 words of final phrase. Always displayed. Always red. Always pop animation.

Sequence Memory Rules

1. **Anti-Repetition:** No effect (zoom_blur, glitch, push_in) may repeat within 15 seconds.
2. **Intensity Budget:** Cumulative average energy over 20-second window. If > 0.7 , force visual-only breathe.
3. **Breathe Trigger:** Budget depleted OR 20+ seconds of high energy. Reset budget to 1.0 after breathe.
4. **Breathe Visual:** Slow push-in ($1.0\times \rightarrow 1.15\times$ over 2s). Music swell (+3 dB). No text.

SFX Placement Rules

1. **Cut SFX:** Every cut gets whoosh_fast (hard) or whoosh_slow (zoom_blur).
2. **Red Text SFX:** Every high-energy word gets impact_deep (body) or impact_sharp (CTA).
3. **Glitch SFX:** Every glitch animation gets glitch_digital.
4. **Pop SFX:** Every pop animation gets pop_text.
5. **Beat Drop SFX:** Energy > 0.85 beats get sub_drop (seeded 50% chance).
6. **Pre-Breathe SFX:** 1 second before breathe, riser_short builds tension.

27. THE SEQUENCE MEMORY STATE MACHINE (NEW)

States

State	Description	Entry Condition	Exit Condition
calm	Low energy, sparse text, no motion	avgEnergy < 0.3 for 5s	avgEnergy > 0.5
building	Energy rising, text density increasing	avgEnergy > 0.5	avgEnergy > 0.7 OR intensityBudget < 0.3
saturated	Max intensity, must breathe	intensityBudget ≤ 0 OR avgEnergy > 0.7 for 20s	breathe completed AND avgEnergy < 0.4
recovering	Post-breathe, energy declining	breathe completed	avgEnergy < 0.3 for 5s

State Transitions

calm **■**(energy rises)**■**> building
building **■**(energy peaks)**■**> saturated
saturated **■**(breathe forced)**■**> recovering
recovering **■**(energy settles)**■**> calm
building **■**(energy drops)**■**> calm

Outputs by State

State	allow_hard_cut	require_breathe	suppress_text	reduce_shake	permit_zoom_blur
calm	yes	no	no	yes	no
building	yes	no	no	no	yes
saturated	no	yes	yes	yes	no
recovering	yes	no	partial	yes	no

Inputs

- Rolling energy (20-second window)
- Time since last breathe
- CTA proximity (within last 3 seconds)
- Repetition score (from Replay Ledger)

28. THE PROMPT GOVERNANCE RULES (NEW)

The Problem

"Can we create the whole architecture and then mess it up with one not well considered prompt?" Yes.

The Rules

1. **Prompt may bias:** doctrine, density, tone, exclusions, style preferences

- 2. **Prompt may NOT override:** determinism contract, variation key logic, render infrastructure
- 3. **Prompt may NOT bypass:** Judgment Layer quality floors, anti-repetition thresholds
- 4. **Prompt may NOT force:** hidden global changes to render pipeline, asset resolution, or bundle behavior
- 5. **Prompt interpretation belongs in:** Planning Snapshot, not in ad hoc Director branches
- 6. **Prompt changes require:** review if they affect Judgment Layer policy or variation contract

Enforcement

- Prompt fingerprint is part of VariationKey
- Prompt changes that affect infrastructure are rejected at validation layer
- Prompt changes that affect editorial policy are passed to Judgment Layer

29. THE EVIDENCE PRESERVATION PATH (NEW)

The Problem

We rejected IRL for Phase 1, but we must preserve the artifacts that make future IRL possible.

What to Persist (Per Output)

Artifact	Format	Purpose
Candidate Treatment Genomes	JSON array	Training data for future policy learning
Chosen candidate	JSON	Ground truth for reward function
Rejected candidates	JSON array	Negative examples for discriminator
Review verdicts	JSON	Human preference signals for RLHF
Failure taxonomy tags	CSV	Supervised learning labels
Sequence-level notes	JSON	Context for temporal reasoning
Planner Audit	JSON	Explainability and debugging

Storage

```
~/.prometheus/evidence/  
  candidates/  
  ■   {jobId}.candidates.json  
  ■   verdicts/  
  ■   {jobId}.verdict.json  
  ■   failures/  
  ■   {jobId}.failures.csv  
  ■   audits/  
  ■   {jobId}.audit.json
```

Why This Matters

This is the real bridge to Phase 2/3 learning, not just the phrase "Phase 4 IRL." Without evidence, you cannot train a discriminator, a reward model, or a preference model.

30. THE CI/CD SPECIFICATION

GitHub Actions Workflow

```
name: Prometheus CI
on:
  push:
  branches: [main, develop]
  pull_request:
  branches: [main]

jobs:
  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with:
      node-version: '20'
      - run: npm ci
      - run: npm run build --workspace=packages/shared-types
      - run: npm run typecheck --workspace=packages/shared-types
      - run: npm run typecheck --workspace=backend
      - run: npm run typecheck --workspace=apps/worker
      - run: npm run typecheck --workspace=remotion-app

  test-backend:
    runs-on: ubuntu-latest
    needs: typecheck
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with:
      node-version: '20'
      - run: npm ci
      - run: npm run test --workspace=backend

  schema-sync:
    runs-on: ubuntu-latest
    needs: typecheck
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with:
      node-version: '20'
      - run: npm ci
      - run: npm run build --workspace=packages/shared-types
      - name: Verify dist matches src
      run: |
        diff -q packages/shared-types/src/unified-render-manifest.ts packages/shared-types/dist/unified-render-manifest.js || exit 1

  determinism:
    runs-on: ubuntu-latest
    needs: [typecheck, schema-sync]
```

```

timeout-minutes: 15
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
  with:
    node-version: '20'
  - run: npm ci
  - run: npm run build --workspace=packages/shared-types
  - name: Same-key determinism
  run: |
    npx tsx scripts/test-joseph.ts --quick --hash ci-test-same
    HASH1=$(sha256sum output/joseph/edit_0_joseph_aggressive.mp4 | cut -d' ' -f1)
    npx tsx scripts/test-joseph.ts --quick --hash ci-test-same
    HASH2=$(sha256sum output/joseph/edit_0_joseph_aggressive.mp4 | cut -d' ' -f1)
    if [ "$HASH1" != "$HASH2" ]; then echo "Determinism failed!"; exit 1; fi
  - name: Different-key variation
  run: |
    npx tsx scripts/test-joseph.ts --quick --hash ci-test-diff
    HASH3=$(sha256sum output/joseph/edit_0_joseph_aggressive.mp4 | cut -d' ' -f1)
    if [ "$HASH1" = "$HASH3" ]; then echo "Variation failed!"; exit 1; fi
  - name: Verify no zombies
  run: |
    sleep 5
    if pgrep -f chrome; then echo "Zombie Chrome!"; exit 1; fi

```

ESLint Configuration

```

{
  "rules": {
    "no-restricted-globals": ["error", "Math", "Date", "performance"],
    "no-restricted-properties": [
      "error",
      {
        "object": "Math",
        "property": "random",
        "message": "Use seededRandom from @prometheus/shared-types"
      }
    ],
    "no-restricted-imports": [
      "error",
      {
        "paths": [
          {
            "name": "gsap",
            "message": "GSAP is forbidden in render path. Use pre-computed physics."
          }
        ]
      }
    ]
  }
}

```

31. THE REVIEW FLOOR (NEW)

The Problem

The "85% Joseph" claim is not measurable and invites self-deception.

The Replacement

Explicit acceptance gates tied to Failure Taxonomy and review:

Gate	Metric	Threshold	Measurement
Rhythm Lock	Cuts perceived as intentional	>90% of review comparisons	Human reviewer scores 1–5
Motion Premium	Text motion judged premium vs. cheap	>85% of comparisons	A/B against CapCut auto-captions
Anti-Fatigue	Sequence does not trigger repetition-fatigue	>99% of runs	3 reviewers, majority vote
Readability	Text readable in >95% of frames	<5% readability-sacrifice tags	Automated + human review
Variation	Same-source re-upload yields distinguishable distance	>95% distance	SHA256 comparison + human review
Audio Polish	Ducking and SFX perceived as professional	>90% of comparisons	Blind listening test
Retention	Breathe beats improve perceived pacing	>80% of reviewers prefer with breathe	A/B test with/without breathe

Why This Matters

Measurable gates prevent "vibe-based" acceptance. They force the system to prove its quality, not assert it.

32. THE VERIFIED ASSUMPTIONS (ACCEPTED RISKS)

These are known risks we accept for MVP. They are documented, not debated.

#	Assumption	Risk	Acceptance
1	Band-pass sidechain has false positive audio symptoms	Audio symptoms incorrectly on some tracks	Documented in `FAILURE_TAXONOMY.md`. Phase 2: Derivative synthesis
2	DOM canvas text fallback is slower than MediaPipe	Artificially drop frames	Acceptable. Render path is identical. Preview can be still-frame
3	1 concurrent render per machine	90-sec video takes 5 min on software	Acceptable for MVP testing. Phase 2 adds cloud GPU.
4	Upper-middle heuristic fails on walking gestures	Openings overlap hands	Acceptable for <5% of frames. Phase 2: Mediapipe.
5	AFINN misses sarcasm	Breathe beat may land on wrong moment	Acceptable. Sarcasm usually has high energy anyway.
6	90-second cap excludes long-form content	Market is Shorts. Long-form is Phase 3	Correct scope.
7	No depth compositing in MVP	Text cannot slide behind speaker	Acceptable. 2D kinetic text is still sellable.
8	No motion blur in MVP	Fast moves look slightly digital	Acceptable. Most viewers won't notice on mobile.
9	No gamification in MVP	No counters, progress bars, emoji	Acceptable. Core edit must work first.
10	No cloud backend in MVP	Runs locally only	Acceptable. Design for cloud, deploy locally.
11	No source separation in MVP	Music and voice are mixed	Acceptable. Band-pass is "good enough."
12	5-minute software render is acceptable	Users wait 5 min for 90-sec video	Acceptable for developer MVP. Phase 2: GPU cloud.
13	2-week sprint is feasible with 2 agents	May require scope cuts	Acceptable. Cut SFX variations first if behind.

#	Assumption	Risk	Acceptance
14	Chrome headless is stable for 90-sec renders	Crash on long renders	Mitigated by 5-min timeout.
15	R3F + Remotion can render 1080×1920 at 30fps	GPU memory may exhaust	Mitigated by GPU probe + swangle fallback.
16	Candidate generation adds <100ms	N candidate × manifest generation	Bounded set (2–6). Pre-computed.
17	Replay Ledger SQLite is sufficient for MVP	Slow with 10k+ entries	SQLite handles 10k rows trivially. Phase 2: migrate if needed.
18	Variation key prevents clone outputs	Same source + same key = same output	This is the contract. Different key = variation.

33. APPENDIX A: GLOSSARY

Term	Definition
R3F	React Three Fiber. React renderer for Three.js.
Remotion	React-based video rendering framework.
CFR	Constant Frame Rate. Every frame has exact duration.
VFR	Variable Frame Rate. Frame durations vary. Forbidden.
CRF	Constant Rate Factor. Quality-based encoding (18 = high quality).
LUFS	Loudness Units Full Scale. Standard for audio loudness.
SFX	Sound Effects.
Ducking	Reducing music volume when voice speaks.
Sidechain	Using one audio signal (voice) to control another (music).
Float32Array	Typed array of 32-bit floats. Zero GC overhead.
SDF	Signed Distance Field. Method for rendering crisp text in WebGL.
Troika	Library for rendering text in Three.js.
Determinism	Same inputs produce identical outputs.
Seed	Initial value for pseudo-random number generator.
PRNG	Pseudo-Random Number Generator.
Breathe Beat	Moment of visual calm (no text) to reset viewer attention.
CTA	Call To Action. Final segment urging viewer action.
Hook	First 3 seconds. Must capture attention.
Thesis Word	Semantically important word (long, confident, high energy).
Stop Word	Common word with low semantic value ("the", "and", "is").
Chunk	Segment of video rendered independently. Not used in MVP.
Stitch	Concatenating chunks into final video. Not used in MVP.

Term	Definition
GAIL	Generative Adversarial Imitation Learning. Phase 4 RL.
IRL	Inverse Reinforcement Learning. Phase 4.
MVP	Minimum Viable Product. What ships in 2 weeks.
Governor	The module with final authority over quality and novelty.
Judgment Layer	The governor. Applies quality floor, anti-repetition, taste.
Top-Level Planner	Generates candidate Treatment Genomes.
Director Adapter	Concrete implementation of Planner + Judgment in Phase 1.
Replay Ledger	Local persisted memory of prior outputs for anti-repetition.
Variation Key	Explicit input (`uploadInstanceId` + `retryIndex`) that causes variation.
Candidate Set	Multiple Treatment Genomes generated per input.
Evidence Preservation	Persisting candidates, rejections, and verdicts for future learning.
Prompt Governance	Rules preventing prompts from breaking infrastructure or determinism.
Review Floor	Measurable acceptance gates replacing vague quality claims.
Sequence Memory	State machine tracking intensity and enforcing breathe beats.
Asset Contract	Explicit separation of browser-facing vs. FFmpeg-facing media references.
Render Entry Contract	Joseph-specific render entry to avoid bundling unrelated app code.

34. APPENDIX B: PLAIN-ENGLISH BREAKDOWN

1. The Big Picture: What Did We Just Design?

At its core, **Prometheus is not a video editor like Premiere Pro or CapCut.**

Standard video editors make *you* do the work. You drag text, you place keyframes, you guess where the audio beat is.

Prometheus is a **"Cinematic Attention Orchestration System."** It is an engine that listens to the audio, reads the transcript, feels the energy of the speaker, and then *mathematically dictates* where cuts should happen, how the text should bounce, and when sound effects should fire.

We cut out all the complicated cloud infrastructure (queues, servers, distributed computing) for this MVP. The v8.1 architecture is a **local, heavy-duty engine** that runs entirely on your machine. You feed it raw footage, and it spits out a highly kinetic, 90-second vertical Short.

2. How the Engine Works (Step-by-Step)

Imagine the architecture as a factory assembly line with four distinct stations:

Station 1: The Observer (Audio & Transcript Analysis)

Before any video is rendered, the system "listens" to the file. It isolates the human voice using a frequency filter (300Hz-3000Hz). It finds the exact milliseconds where the speaker gets loud, where the music drops, and where the natural pauses are. It maps the energy of the video frame by frame.

Station 2: The Director (The Rule Engine)

This is the brain. The Director looks at the audio map and the transcript and starts making decisions based on predefined profiles (Aggressive, Cinematic, Minimal).

- *"The speaker screamed here? Make the text huge, red, and shake the camera."*
- *"There is a heavy music beat here? Force a hard cut."*
- *"The energy has been crazy for 15 seconds? Force a 'breathe beat'—stop the text, slow push-in on the face, let the music swell."*

Station 3: The Physics Engine (The Math)

In cheap editors, text just "pops" or "fades" in. In Joseph edits, text feels like it has weight and tension. Our physics engine calculates the exact X/Y coordinates, scale, and rotation for every word on every frame using spring mathematics.

The Secret Sauce: We do all this math upfront in Node.js (the CPU), not during the video render. It creates a massive spreadsheet of instructions.

Station 4: The Renderer (React Three Fiber + Remotion)

The renderer (running in a hidden, automated Chrome browser) reads the spreadsheet. It does not have to think; it just draws exactly what the Physics Engine tells it to. It composites the 3D text over the video, applies the shaders (like Zoom Blurs), ducks the music volume perfectly under the voice, and exports the final MP4.

3. What This Architecture Fixes (The Hidden Traps)

Why did we go through 8 versions to get here? Because building automated video at scale is a minefield. Here is what v8.1 protects us from:

- **The "Random Drift" Problem (Determinism):** If you render a video twice with the same key, it **MUST** look exactly the same both times. If you rely on the computer's graphics card to do math, or use standard animation tools, things drift by a few pixels or milliseconds. V8.1 locks the math to the CPU. 100% predictable.
- **The "Melted Laptop" Problem:** By strictly capping the video at 90 seconds and doing a single sequential render, we avoid spawning 10 invisible Chrome browsers that would crash your computer.
- **The "Messy Audio" Problem:** If you just lower the music volume whenever the audio track gets loud, a loud drum beat in the background music will actually trigger the ducking, causing the audio to "pump" weirdly.

V8.1 filters the audio so it *only* ducks when a human voice speaks.

- **The "TikTok Swipe" Problem:** If a video is 100% kinetic energy non-stop, viewers get exhausted and swipe away. V8.1's "Sequence Memory" tracks the intensity and forces visual "breathes" to keep human retention high.
- **The "Clone Output" Problem:** If you upload the same video twice, a cheap system would give you the exact same edit. V8.1's **Variation Contract** ensures that every upload gets a different (but equally high-quality) Treatment Genome, selected by the **Judgment Layer** from a pool of candidates.
- **The "Prompt Sabotage" Problem:** One bad prompt could break the whole system. V8.1's **Prompt Governance Rules** prevent prompts from overriding infrastructure, determinism, or quality floors.

4. The Ultimate Question: Will it look like Joseph?

If we implement v8.1 perfectly, **Yes... with one asterisk.**

It will hit about **85% of the Joseph aesthetic**. It will look undeniably premium, highly engaging, and leagues above standard auto-captioning tools like OpusClip or Veed.

What We WILL Nail Perfectly (The 85%):

1. **The Kinetic Rhythm:** The text won't just appear; it will land, bounce, and react to the speaker's vocal energy with mathematical precision.
2. **The Musical Lock:** Because we force cuts to land strictly on audio transients (beats), the edit will feel deeply musical and intentionally paced.
3. **The Audio Polish:** The sound effects (whooshes, risers, impacts) will be perfectly synced and varied, and the music ducking will sound professional.
4. **The Restraint:** Because of the Sequence Memory, the video won't look like a chaotic mess. It will know when to be aggressive and when to back off.
5. **The Variation:** Because of the Candidate Set + Replay Ledger + Judgment Layer, every re-upload will feel fresh, not cloned.

The Asterisk: What We Are Missing in Phase 1 (The Final 15%):

To get the MVP out the door in two weeks and ensure it doesn't crash, we intentionally deferred a few "God-tier" Joseph features:

- **Depth Occlusion (Text behind the speaker):** True Joseph edits cut the speaker out of the background so text can slide behind their head. This requires heavy AI computer vision (RVM Matting). In v8.1, text is purely overlaid on top (though we avoid the face).
- **Semantic "Ironic" Cuts:** Joseph sometimes cuts to a weird angle or holds on a silent, awkward facial expression for comedic effect. V8.1 is driven by audio energy and rhythm, so it can't understand nuanced visual irony yet.
- **B-Roll and Gamification:** Joseph uses sound-effect-driven progress bars, emoji replacements, and cutaway memes. V8.1 is purely focused on the speaker, the typography, and the camera moves.

The Verdict

When you hit export on v8.1, you will get a video that makes an average viewer say, *"Wow, you hired a really good editor for this."*

It won't have the handcrafted, meme-heavy, 3D-depth perfection of a true \$500/video Joseph edit—but it will have the **rhythm, the physics, the polish, and the variation** that defines his core style. And it will generate it automatically in minutes.

That is what v8.1 buys you.

SIGN-OFF

This document is **LOCKED** for build.

- **Architecture:** v8.1 — Local-First Doctrine + Codex Amendments
- **Paradox Status:** Resolved (distributed infrastructure cut for MVP)
- **Sprint:** 14 days, 2 agents, 19 tickets
- **Stack:** R3F + Remotion + FFmpeg
- **Render Mode:** Sequential local CLI
- **Target:** 90-second vertical Shorts (1080×1920)
- **Governor:** Judgment Layer
- **Variation:** Candidate Treatment Genomes + Replay Ledger + Explicit Variation Key
- **Go/No-Go Gates:** 8 checkpoints + 2 variation gates
- **Phase 2 Hook:** renderFromManifest() signature preserved for BullMQ wrapping

Approved by: Gemini (Strategic Review) + Kimi (Architecture) + Codex (Grill & Amendments) + Human (Final Authority)

Date: 2026-06-20

Next Action: Execute Step 1 (Bundle Cache). Begin 14-day sprint.

"The paradox is dead. The governor is named. The variation is explicit. The build begins now."